

Lists

Recap

- Tuples are indexable ordered sequences of objects
 - denoted by parentheses, `()`
 - **immutable**, we cannot change the values of specific elements

Today, we look at a very similar data type, but **mutable**

Lists

Lists

- *Indexable ordered sequence* of objects
 - usually one type
 - can be mixed but not recommended
- Denoted by square brackets, `[]`
- Compared to tuples which are denoted by parentheses, `()`

Mutable

This means that we can change the values of specific elements

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists

```
1  a_list = [] # empty list
2  b_list = [2, 'A', 4, [1, 2]] # list of mixed types
3  [1, 2] + [3, 'four'] # evaluates to [1, 2, 3, 'four']
4
5  len(b_list)
6  b_list[0]
7  b_list[2] + 1
8  b_list[3]
9  b_list[4]
10 i = 2
11 b_list[i-1]
```

Lists are mutable

Unlike a tuple, we can assign directly into an index of a list

```
1  b_list = [2, 'A', 4, [1, 2]]
2  b_list[0] = 100 # allowed! lists are mutable
3  print(b_list) # [100, 'A', 4, [1, 2]]
4
5  ts = (1, 2, 3)
6  ts[0] = 100 # TypeError, tuples are immutable
```

This is the key difference between lists and tuples: a list can be changed *in place* after it's created, a tuple cannot

Iterating over a list

For example, we want to compute the sum of all elements in a list

```
1 total = 0
2 for i in range(len(list_a)):
3     total = total + list_a[i]
4 print(total)
```

Because working with every single item of a list is very common, we have

```
1 total = 0
2 for item in list_a:
3     total = total + item
4 print(total)
```

Note:

- list elements are indexed from 0 to `len(list)-1`
- `range` goes from 0 to `n-1`

Iterating over a list

Both loops on the previous slide produce the exact same result

- the first version needs an index `i` to look up each element manually, `list_a[i]`
- the second version lets Python hand you each `item` directly, no index bookkeeping needed

Prefer the `for item in list_a:` style whenever you don't actually need the index itself

- but if you need to know *where* an element is (its position), you'll still want `range(len(list_a))` or `enumerate(list_a)`